CSE 208: Data Structures and Algorithms II Sessional
Performance Analysis on Hashing Methods

Md. Fuad Al Alam
Student ID: 2205167

July 07, 2025

Hash Functions Used

Two distinct hash functions, `hash1` and `hash2`, are implemented. Both are polynomial rolling hash functions, which are commonly used for strings due to their simplicity and good distribution properties. They are designed to minimize collisions by using different prime bases.

- `hash1(const string& key)`: This function calculates a hash value using a polynomial rolling hash approach with a base `BASE1` and a modulus `MOD` (which is the table size).

$$H_1(k) = \left(\sum_{i=0}^{L-1} (k_i -' a' + 1) \cdot \text{BASE1}^i \right) \pmod{\text{MOD}}$$

  where k_i is the i-th character of the key, and L is the length of the key.

- `hash2(const string& key)`: Similar to `hash1`, this function also uses a polynomial rolling hash, but with a different prime base `BASE2` to provide a distinct hash value, which is crucial for double hashing.

$$H_2(k) = \left(\sum_{i=0}^{L-1} (k_i -' a' + 1) \cdot \text{BASE2}^i \right) \pmod{\text{MOD}}$$

  The bases `BASE1` and `BASE2` are randomly selected prime numbers (e.g., 29, 31, 37, 41) at initialization to ensure they are distinct. The use of different prime bases helps in achieving better distribution and reducing the likelihood of keys colliding under both hash functions simultaneously.

Collision Resolution Techniques

The `VaultXHashTable` supports three collision resolution techniques:

1. **Separate Chaining with Balanced BSTs (Red-Black Trees):** Each slot in the hash table points to a Red-Black Tree. When a collision occurs, the new key-value pair is inserted into the Red-Black Tree at the corresponding hash table

index. This ensures that search, insertion, and deletion operations within a chain remain efficient ($O(\log k)$ where k is the number of elements in the chain) due to the balanced nature of the BST.

2. **Linear Probing with Step Adjustment:** The probing sequence is defined by the formula:
$$\text{linearHash}(k, i) = (\text{Hash}(k) + i \times S) \pmod N$$
where `Hash(k)` is either `hash1(k)` or `hash2(k)`, i is the current probe count, and S is a small prime step size (e.g., 5 in this implementation), chosen at initialization. This method probes linearly through the table with a fixed step.

3. **Double Hashing:** The probing sequence is defined by the formula:
$$\text{DoubleHash}(k, i) = (\text{Hash1}(k) + i \times \text{Hash2}(k)) \pmod N$$
where `Hash1(k)` and `Hash2(k)` are the two distinct hash functions, and i is the current probe count. This method uses a second hash function to determine the step size for probing, which helps in reducing clustering compared to linear probing.

Constants Used

- `word_count`: 10000 (Total number of random words generated)
- `table_size`: 10000 (Initial size of the hash table, adjusted to the nearest prime)
- `BASE1`, `BASE2`: Randomly generated prime numbers (e.g., from {29, 31, 37, 41}) at runtime for hash functions.
- `MOD`: The actual prime `tableSize` used as the modulus for hash functions.
- `stepSize` **(for Linear Probing)**: 5 (A small prime number).

Methodology

Word Generation

10,000 unique random words (5-10 lowercase letters) were generated once at the beginning.

Load Factor Variation

The load factor was varied from 0.4 to 0.9 in steps of 0.1. For each load factor a, the number of elements to insert was $a \times$ table_size.

Insertion

Elements were inserted into the hash table using each collision resolution method and both hash functions. The total number of collisions during insertion was recorded.

Search Test (Before Deletion)

10% of the currently inserted elements were randomly selected and searched. The average search time and average number of probes (for open addressing) were measured.

Deletion

10% of the inserted elements were randomly selected and deleted from the hash table.

Search Test (After Deletion)

A new search set of the same size was constructed. Half of these elements were from the deleted set (expected not to be found), and the other half were still present in the table (expected to be found). The average search time and average number of probes (for open addressing) were measured again.

Results

The following tables present the performance metrics for each load factor, comparing Hash1 and Hash2 functions across the three collision resolution methods. All times are in microseconds (μs).

Performance for Load Factor: 0.4

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	731	0.09	N/A	0.12	N/A	709	0.09	N/A	0.15	N/A
LINEAR_PROBING	1374	0.10	1.25	0.17	2.51	1342	0.10	1.36	0.17	2.62
DOUBLE_HASHING	1188	0.15	1.36	0.20	2.15	1088	0.15	1.28	0.24	2.09

Performance for Load Factor: 0.5

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	1080	0.09	N/A	0.19	N/A	1078	0.09	N/A	0.12	N/A
LINEAR_PROBING	2424	0.11	1.47	0.18	2.77	2662	0.11	1.49	0.16	3.06
DOUBLE_HASHING	1907	0.15	1.36	0.27	2.47	1958	0.16	1.37	0.25	2.36

Performance for Load Factor: 0.6

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	1519	0.10	N/A	0.19	N/A	1522	0.09	N/A	0.11	N/A
LINEAR_PROBING	4625	0.12	1.82	0.19	3.48	4855	0.11	1.75	0.17	4.13
DOUBLE_HASHING	3292	0.15	1.47	0.24	2.67	3161	0.16	1.53	0.20	2.72

Performance for Load Factor: 0.7

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	1955	0.09	N/A	0.18	N/A	1976	0.09	N/A	0.11	N/A
LINEAR_PROBING	8772	0.12	2.27	0.20	6.12	8772	0.12	2.37	0.20	6.29
DOUBLE_HASHING	5125	0.16	1.82	0.23	3.30	5125	0.18	1.73	0.22	3.58

Performance for Load Factor: 0.8

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	2544	0.09	N/A	0.16	N/A	2519	0.09	N/A	0.09	N/A
LINEAR_PROBING	15300	0.13	2.69	0.26	9.66	17379	0.13	3.21	0.26	10.86
DOUBLE_HASHING	8144	0.17	2.05	0.23	4.32	8044	0.17	1.97	0.23	4.74

Performance for Load Factor: 0.9

Method	Hash1 Function					Hash2 Function				
	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes	# Collisions	Avg Time (μs)	Avg Probes	Post-Del Time (μs)	Post-Del Probes
CHAINING	3133	0.09	N/A	0.16	N/A	3057	0.09	N/A	0.13	N/A
LINEAR_PROBING	41147	0.17	5.39	0.53	30.02	49401	0.21	7.73	0.55	33.03
DOUBLE_HASHING	24578	0.17	2.50	0.29	7.49	14229	0.19	2.64	0.26	7.41

Impact of Load Factors on Results

Based on the derived results, the impact of load factors on the performance metrics can be observed:

- **Collisions during Insertion:** As load factor increases, collisions consistently rise for all methods. Open addressing methods, especially Linear Probing, show significant increases at higher load factors.
- **Average Search Time:** Average search time generally increases with load factor for open addressing. Chaining maintains very low search times across all load factors due to balanced BSTs.
- **Average Probes (Open Addressing):** Probes significantly increase with higher load factors. Double Hashing generally performs better by mitigating clustering, but both show increased probes after deletion due to tombstones.
- **Impact of Deletion:** Deletion has minimal impact on Separate Chaining's search performance. For open addressing, both search time and probes increase significantly after deletion, especially at higher load factors.
- **Hash Function Comparison (Hash1 vs. Hash2):** Both hash functions perform comparably with minor variations in collisions and probes. No single hash function consistently outperforms the other across all scenarios.

Conclusion

The implemented `VaultXHashTable` effectively demonstrates the characteristics of various hashing techniques. The polynomial rolling hash functions, combined with prime table sizes and distinct bases, generally provide good distribution.

The experimental results clearly show that:

1. **Collisions increase with load factor** for all methods, as expected.
2. **Separate Chaining with Red-Black Trees** consistently provides very fast search times, largely unaffected by increasing load factors or deletions, due to the $O(\log k)$ complexity of BST operations within each chain.

3. **Open Addressing methods (Linear Probing and Double Hashing)** experience a significant increase in average probes and search times as the load factor rises, particularly after deletions.

4. **Double Hashing generally performs better than Linear Probing** in terms of average probes, especially at higher load factors, indicating its effectiveness in mitigating primary clustering.

5. The **impact of deletions is more pronounced on open addressing methods** due to the need to mark slots, which increases subsequent search probe counts.

In summary, for scenarios requiring consistent and high performance across varying load factors and dynamic operations (insertions, searches, deletions), Separate Chaining with Balanced BSTs is the preferred approach. For open addressing, Double Hashing offers a better balance of performance compared to Linear Probing, particularly at higher loads.